JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class AbstractQueue<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractQueue<E>

Type Parameters:

`E - the type of elements held in this collection`

All Implemented Interfaces:

`Iterable<E>, Collection<E>, Queue<E>`

Direct Known Subclasses:

`ArrayBlockingQueue, ConcurrentLinkedQueue, DelayQueue, LinkedBlockingDeque, LinkedBlockingQueue, LinkedTransferQueue, PriorityBlockingQueue, PriorityQueue, SynchronousQueue`

```
public abstract class AbstractQueue<E>
extends AbstractCollection<E>
implements Queue<E>
```

This class provides skeletal implementations of some `Queue` operations. The implementations in this class are appropriate when the base implementation does *not* allow `null` elements. Methods `add`, `remove`, and `element` are based on `offer`, `poll`, and `peek`, respectively, but throw exceptions instead of indicating failure via `false` or `null` returns.

A `Queue` implementation that extends this class must minimally define a method `Queue.offer(E)` which does not permit insertion of `null` elements, along with methods `Queue.peek()`, `Queue.poll()`, `Collection.size()`, and `Collection.iterator()`. Typically, additional methods will be overridden as well. If these requirements cannot be met, consider instead subclassing `AbstractCollection`.

This class is a member of the Java Collections Framework.

Since:

1.5

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`AbstractQueue()`	Constructor for use by subclasses.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
boolean	`add(E e)`	Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and throwing an `IllegalStateException` if no space is currently available.
boolean	`addAll(Collection<? extends E> c)`	Adds all of the elements in the specified collection to this queue.
void	`clear()`	Removes all of the elements from this queue.
`E`	`element()`	Retrieves, but does not remove, the head of this queue.
`E`	`remove()`	Retrieves and removes the head of this queue.

Methods inherited from class java.util.AbstractCollection

`contains`, `containsAll`, `isEmpty`, `iterator`, `remove`, `removeAll`, `retainAll`, `size`, `toArray`, `toArray`, `toString`

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.Queue

offer, peek, poll

Methods inherited from interface java.util.Collection

contains, containsAll, equals, hashCode, isEmpty, iterator, parallelStream, remove, removeAll, removeIf, retainAll, size, spliterator, stream, toArray, toArray

Methods inherited from interface java.lang.Iterable

forEach

Constructor Detail

AbstractQueue

```
protected AbstractQueue()
```

Constructor for use by subclasses.

Method Detail

add

```
public boolean add(E e)
```

Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and throwing an `IllegalStateException` if no space is currently available.

This implementation returns `true` if `offer` succeeds, else throws an `IllegalStateException`.

Specified by:

`add` in interface `Collection<E>`

Specified by:

`add` in interface `Queue<E>`

Overrides:

`add` in class `AbstractCollection<E>`

Parameters:

`e` - the element to add

Returns:

`true` (as specified by `Collection.add(E)`)

Throws:

`IllegalStateException` - if the element cannot be added at this time due to capacity restrictions

`ClassCastException` - if the class of the specified element prevents it from being added to this queue

`NullPointerException` - if the specified element is null and this queue does not permit null elements

`IllegalArgumentException` - if some property of this element prevents it from being added to this queue

remove

```
public E remove()
```

Retrieves and removes the head of this queue. This method differs from `poll` only in that it throws an exception if this queue is empty.

This implementation returns the result of `poll` unless the queue is empty.

Specified by:

`remove` in interface `Queue<E>`

Returns:

the head of this queue

Throws:

`NoSuchElementException` - if this queue is empty

element

```
public E element()
```

Retrieves, but does not remove, the head of this queue. This method differs from `peek` only in that it throws an exception if this queue is empty.

This implementation returns the result of `peek` unless the queue is empty.

Specified by:

`element` in interface `Queue<E>`

Returns:

the head of this queue

Throws:

`NoSuchElementException` - if this queue is empty

clear

```
public void clear()
```

Removes all of the elements from this queue. The queue will be empty after this call returns.

This implementation repeatedly invokes `poll` until it returns `null`.

Specified by:

`clear` in interface `Collection<E>`

Overrides:

`clear` in class `AbstractCollection<E>`

addAll

```
public boolean addAll(Collection<? extends E> c)
```

Adds all of the elements in the specified collection to this queue. Attempts to addAll of a queue to itself result in `IllegalArgumentException`. Further, the behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

This implementation iterates over the specified collection, and adds each element returned by the iterator to this queue, in turn. A runtime exception encountered while trying to add an element (including, in particular, a `null` element) may result in only some of the elements having been successfully added when the associated exception is thrown.

Specified by:

`addAll` in interface `Collection<E>`

Overrides:

`addAll` in class `AbstractCollection<E>`

Parameters:

c - collection containing elements to be added to this queue

Returns:

true if this queue changed as a result of the call

Throws:

`ClassCastException` - if the class of an element of the specified collection prevents it from being added to this queue

`NullPointerException` - if the specified collection contains a null element and this queue does not permit null elements, or if the specified collection is null

`IllegalArgumentException` - if some property of an element of the specified collection prevents it from being added to this queue, or if the specified collection is this queue

`IllegalStateException` - if not all the elements can be added at this time due to insertion restrictions

See Also:

`add(Object)`

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.